

# Missing Rate Limiting

# 1. Missing Rate Limiting on Contact Form API

## OWASP Category

A04:2021 – Insecure Design

## Affected Files

nginx configuration  
/etc/nginx/sites-available/lt-nilavan

Application endpoint:  
/api/sendgrid

## Severity

High

## Description

The /api/sendgrid endpoint s unlimited POST requests without any rate limiting or abuse prevention mechanism. The vulnerability exists because no request throttling controls were implemented at the application or reverse proxy level.

## Business Impact

An attacker can flood the business owner's inbox with spam emails, exhaust the SendGrid email quota, cause operational disruption, and increase infrastructure costs.

## Proof of Concept

```
for i in $(seq 1 20); do
curl -X POST https://nilvan.duckdns.org/api/sendgrid \
-H "Content-Type: application/json" \
-d
'{"name":"spam","email":"test@test.com","phone":"9999999999","message":"
spam"}'
done
```

## Result

All requests were accepted successfully without any restriction.

```
siva-leadtap@ip-10-0-10-55:/etc/nginx/sites-available$ for i in $(seq 1 20); do
curl -X POST https://nilvan.duckdns.org/api/sendgrid \
-H "Content-Type: application/json" \
-d '{"name":"spam","email":"test@test.com","phone":"999999999","message":"spam message"}'
echo ""
done
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
siva-leadtap@ip-10-0-10-55:/etc/nginx/sites-available$
```

|                          |  |    |                                                                                                                          |          |
|--------------------------|--|----|--------------------------------------------------------------------------------------------------------------------------|----------|
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: spam Email: test@test.c...  | 1:04 PM  |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 1:02 PM  |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: spam Email: test@test.c...  | 1:02 PM  |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: spam Email: test@test.c...  | 1:00 PM  |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 1:00 PM  |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 12:58 PM |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 12:58 PM |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 12:58 PM |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 12:58 PM |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 12:57 PM |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 12:56 PM |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@test.co... | 12:56 PM |
| <input type="checkbox"/> |  | me | New Contact Form Submission - NILAVAN REALTORS You have a new contact form submission: Name: test Email: test@tes        | 35 PM    |

**NILAVAN REALTORS**

---

You have a new contact form submission:

**Name:** test

**Email:** [test@test.com](mailto:test@test.com)

**Phone:** 9999999999

**Message:** spam

# Recommended Fix

Implemented nginx rate limiting:

```
limit_req_zone $binary_remote_addr zone=sendgrid_limit:10m rate=2r/m;

location /api/sendgrid {
    limit_req zone=sendgrid_limit burst=1 nodelay;

    proxy_pass http://localhost:3000;
}
```

```
skva-leadtap@ip-10-0-10-55:/etc/nginx/sites-available$ for i in $(seq 1 10); do
curl -i -X POST https://nilvan.duckdns.org/api/sendgrid \
-H "Content-Type: application/json" \
-d '{"name":"test","email":"test@test.com","phone":"9999999999","message":"spam"}'
done
HTTP/2 200
server: nginx
date: Sat, 09 May 2026 07:28:00 GMT
content-type: application/json
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch, Next-Router-Segment-Prefetch
x-frame-options: DENY
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
strict-transport-security: max-age=63072000; includeSubDomains; preload
permissions-policy: geolocation=(), microphone=(), camera=()
content-security-policy: default-src 'self'; base-uri 'self'; object-src 'none'; frame-ancestors 'none'; form-action 'self'; script-src 'self' 'unsafe-inlin
e' 'unsafe-eval' https; style-src 'self' 'unsafe-inline' https; img-src 'self' data: blob: https; font-src 'self' data: https; connect-src 'self' https:
; frame-src 'self' https;;

{"success":true}HTTP/2 200
server: nginx
date: Sat, 09 May 2026 07:28:00 GMT
content-type: application/json
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch, Next-Router-Segment-Prefetch
x-frame-options: DENY
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
strict-transport-security: max-age=63072000; includeSubDomains; preload
permissions-policy: geolocation=(), microphone=(), camera=()
content-security-policy: default-src 'self'; base-uri 'self'; object-src 'none'; frame-ancestors 'none'; form-action 'self'; script-src 'self' 'unsafe-inlin
e' 'unsafe-eval' https; style-src 'self' 'unsafe-inline' https; img-src 'self' data: blob: https; font-src 'self' data: https; connect-src 'self' https:
; frame-src 'self' https;;

{"success":true}HTTP/2 503
server: nginx
date: Sat, 09 May 2026 07:28:00 GMT
content-type: text/html
content-length: 190
x-frame-options: DENY
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
```

```
skva-leadtap@ip-10-0-10-55: /  x  +  v
location ~ /\.(!well-known).* {
    deny all;
    access_log off;
    log_not_found off;
}

# Block .git access
location /.git {
    deny all;
    return 403;
}

# API rate limiting
location /api/ {
    limit_req zone=api_limit burst=1 nodelay;

    proxy_pass http://127.0.0.1:3000;

    proxy_http_version 1.1;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;

    # Mitigate Next.js middleware vulnerability
    proxy_set_header x-middleware-subrequest "";

    proxy_cache_bypass $http_upgrade;
}

# Main app
location / {
    proxy_pass http://127.0.0.1:3000;

    proxy_http_version 1.1;

    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

-- VISUAL --

43 48,7 76%

```
xiva-leadtap@ip-10-0-10-55:/etc/nginx/sites-available$ cat cd ../nginx.conf
user www-data;
worker_processes auto;
worker_cpu_affinity auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log;
include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 768;
    # multi_accept on;
}

http {

    ##
    # Basic Settings
    ##

    sendfile on;
    tcp_nopush on;
    types_hash_max_size 2048;
    #server_tokens build; # Recommended practice is to turn this off

    # server_names_hash_bucket_size 64;
    # server_name_in_redirect off;
    server_tokens off;
    limit_req_zone $binary_remote_addr zone=api_limit:10m rate=2r/m;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    ##
    # SSL Settings
    ##

    ssl_protocols TLSv1.2 TLSv1.3; # Dropping SSLv3 (POODLE), TLS 1.0, 1.1
```

# HTML Injection

## 2. HTML Injection in Contact Form Email Template

### OWASP Category

A03:2021 – Injection

### Affected File & Line Number

[app/api/sendgrid/route.ts](#) Lines 28-44

Severity: High

Affected code:

```
<p><strong>Name:</strong> ${name}</p>
<p><strong>Email:</strong> ${email}</p>
<p><strong>Phone:</strong> ${phone}</p>
<p><strong>Message:</strong> ${message}</p>
```

```
sendgrid.setApiKey(apiKey);

const msg = {
  to: toEmail,
  from: toEmail, // Must be a verified sender in SendGrid
  subject: 'New Contact Form Submission',
  text: `Name: ${name}\nEmail: ${email}\nPhone: ${phone}\nMessage: ${message}`,
  html: `
    <html>
      <body style="background: #f6f6f7; padding: 40px 0;">
        <div style="max-width: 480px; margin: 40px auto; background: #fff; border-radius: 18px; box-shadow: 0 2px 8px rgba(0,0,0,0.04); padding: 32px 32px 24px 32px; font-family: Arial, sans-serif;">
          <div style="text-align: center; margin-bottom: 24px;">
            <div style="font-size: 22px; font-weight: bold; letter-spacing: 1px; color: #222;">NILAVAN REALTORS</div>
          </div>
          <hr style="border: none; border-top: 1px solid #eee; margin: 24px 0;" />
          <div style="font-size: 16px; color: #222; margin-bottom: 24px;">
            <p style="margin: 0 0 16px 0;">You have a new contact form submission:</p>
            <p style="margin: 0 0 8px 0;"><strong>Name:</strong> ${name}</p>
            <p style="margin: 0 0 8px 0;"><strong>Email:</strong> ${email}</p>
            <p style="margin: 0 0 8px 0;"><strong>Phone:</strong> ${phone}</p>
            <p style="margin: 0 0 8px 0;"><strong>Message:</strong> ${message}</p>
          </div>
        </div>
      </body>
    </html>
  `;
};

await sendgrid.send(msg);
return NextResponse.json({ success: true });
} catch (error: unknown) {
  console.error('SendGrid Error:', error);
}
```

21,0-1 65%

### Description

The application directly inserts user-controlled input into the HTML email template without sanitization or escaping. Since the values are rendered inside the HTML body, malicious content such as clickable phishing links or manipulated formatting can be displayed to administrators receiving the email.

### Business Impact

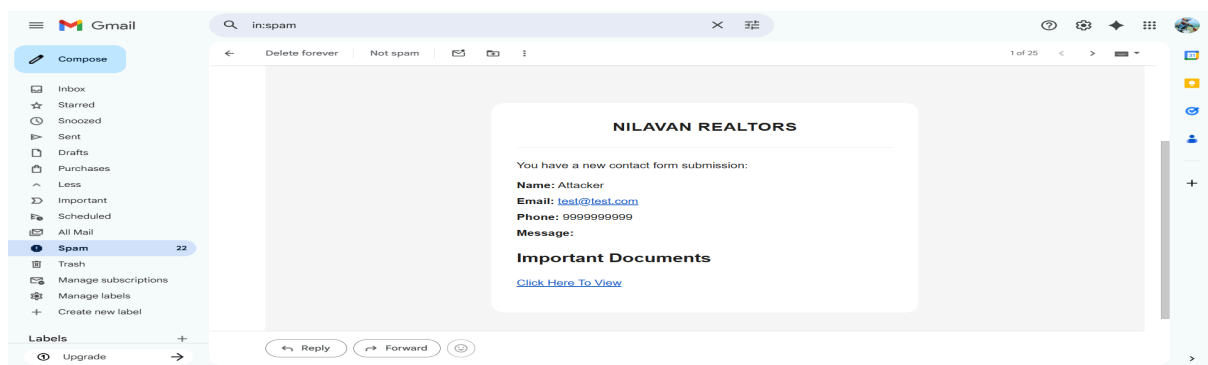
An attacker can:

- Inject malicious HTML into business emails
- Add phishing or scam links
- Manipulate email content viewed by staff
- Spoof trusted-looking messages
- Mislead employees into clicking attacker-controlled URLs

## Proof of Concept

```
curl -X POST https://nilvan.duckdns.org/api/sendgrid \g/api/sendgrid \
-H "Content-Type: application/json" \
-d '{
  "name": "Attacker",
  "email": "test@test.com",
  "phone": "9999999999",
  "message": "<h2>Important Documents</h2><a
href=\"https://example.com\">Click Here To View</a>"
}'
{"success": true}
```

```
siva-leadtap@ip-10-0-10-55:~$ curl -X POST https://nilvan.duckdns.org/api/sendgrid \g/api/sendgrid \
-H "Content-Type: application/json" \
-d '{
  "name": "Attacker",
  "email": "test@test.com",
  "phone": "9999999999",
  "message": "<h2>Important Documents</h2><a href=\"https://example.com\">Click Here To View</a>"
}'
{"success": true}siva-leadtap@ip-10-0-10-55:~$ |
```



## Resulting behaviour:

The received email rendered:

- **HACKED** in bold text
- A clickable external link inside the email body

This confirms that HTML content is executed instead of displayed as plain text.



## Recommended Fix

Escape all user-controlled values before inserting them into HTML.

Example remediation:

```
const escapeHtml = (value: string) =>
  value
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
```

```
// 1. Helper function to prevent HTML Injection
const escapeHtml = (value: string) => {
  if (!value) return "";
  return String(value)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
};
```

Secure usage:

```
<p><strong>Name:</strong> ${escapeHtml(name)}</p>
<p><strong>Message:</strong> ${escapeHtml(message)}</p>
```

```
// 6. Apply escapeHtml() to user variables in the HTML template (HTML Injection Fix)
html: `
  <html>
  <body style="background: #f6f6f7; padding: 40px 0;">
    <div style="max-width: 480px; margin: 40px auto; background: #fff; border-radius: 18px; box-shadow: 0 2px 8px
x rgba(0,0,0,0.04); padding: 32px 32px 24px 32px; font-family: Arial, sans-serif;">
      <div style="text-align: center; margin-bottom: 24px;">
        <div style="font-size: 22px; font-weight: bold; letter-spacing: 1px; color: #222;">NILAVAN REALTORS</div>
      </div>
      <hr style="border: none; border-top: 1px solid #eee; margin: 24px 0;" />
      <div style="font-size: 16px; color: #222; margin-bottom: 24px;">
        <p style="margin: 0 0 16px 0;">You have a new contact form submission:</p>
        <p style="margin: 0 0 8px 0;"><strong>Name:</strong> ${escapeHtml(name)}</p>
        <p style="margin: 0 0 8px 0;"><strong>Email:</strong> ${escapeHtml(email)}</p>
        <p style="margin: 0 0 8px 0;"><strong>Phone:</strong> ${escapeHtml(phone)}</p>
        <p style="margin: 0 0 8px 0;"><strong>Message:</strong> ${escapeHtml(message)}</p>
      </div>
    </body>
  </html>
`
```

# Missing Server-Side Input Validation

# 3. Missing Server-Side Input Validation

## OWASP Category

A05:2021 – Security Misconfiguration

## Affected File & Line Number

[app/api/sendgrid/route.ts](#) Lines 6-7

## Severity: High

Affected code:

```
const body = await req.json();
const { name, email, phone, message } = body;
```

```
import { NextResponse } from 'next/server';
import sendgrid from '@sendgrid/mail';

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const { name, email, phone, message } = body;

    const apiKey = process.env.SENDGRID_API_KEY;
    const toEmail = process.env.SENDGRID_TO_EMAIL;

    if (!apiKey) {
      console.error('SENDGRID_API_KEY is not set!');
      return NextResponse.json({ error: 'SendGrid API key not configured' }, { status: 500 });
    }

    if (!toEmail) {
      console.error('SENDGRID_TO_EMAIL is not set!');
      return NextResponse.json({ error: 'Recipient email not configured' }, { status: 500 });
    }

    sendgrid.setApiKey(apiKey);

    const msg = {
      to: toEmail,
      from: toEmail, // Must be a verified sender in SendGrid
      subject: 'New Contact Form Submission',
      text: `Name: ${name}\nEmail: ${email}\nPhone: ${phone}\nMessage: ${message}`,
      html: ''
    };
  }
}
```

## Description

The /api/sendgrid endpoint does not validate user input on the server side.

The API accepts any JSON data, including:

- Invalid email addresses
- Oversized payloads
- Non-string values
- Malicious content
- Empty fields

Validation currently exists only on the frontend form and can be bypassed easily using direct API requests.

## Business Impact

An attacker can:

- Send malformed requests directly to the API
- Cause email delivery failures
- Generate unnecessary server load
- Flood logs with invalid requests
- Abuse the endpoint using oversized payloads

## Proof of Concept

[illegible][illegible]

NILAVAN REALTORS

You have a new contact form submission:

Name: 12345

Email: not-an-email

Phone: AAAAAAA

Message: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

← Reply

→ Forward

😊

## Resulting behaviour:

The API accepted invalid data and attempted to process the request instead of rejecting it with validation errors.

## Recommended Fix

Implement strict server-side validation using Zod.

Example:

```
import { z } from "zod";

const ContactSchema = z.object({
  name: z.string().min(1).max(80),
  email: z.string().email().max(254),
  phone: z.string().min(7).max(20),
  message: z.string().min(1).max(1000),
});

const parsed = ContactSchema.safeParse(body);

if (!parsed.success) {
  return NextResponse.json(
    { error: "Invalid request" },
    { status: 400 }
  );
}
```

```
// 2. Define strict Zod validation schema
const ContactSchema = z.object({
  name: z.string().min(1, "Name is required").max(80),
  email: z.string().email("Invalid email format").max(254),
  phone: z.string().min(7, "Phone number is too short").max(20),
  message: z.string().min(1, "Message is required").max(2000),
});
```

# Broken Access Control

# No Origin / Referer Validation on Contact Form API

## OWASP Category

A01:2021 – Broken Access Control

## Affected File & Line Number

app/api/sendgrid/[route.ts](#) Line 5

## Severity

Medium

## Description

The /api/sendgrid endpoint accepted requests from any origin without validating trusted frontend domains.

The API did not verify:

- Origin header
- Referer header
- trusted frontend sources

As a result, attackers could directly interact with the API using curl, Postman, scripts, or malicious external websites.

The vulnerability existed because the backend trusted all incoming requests without validating whether they originated from the official application domain.

## Business Impact

An attacker can:

- Abuse the public API from external websites
- Automate spam submissions
- Bypass frontend restrictions
- Trigger unauthorized backend requests

## Proof of Concept

```
curl -X POST https://nilvan.duckdns.org/api/sendgrid \  
-H "Origin: https://evil.com" \  
-H "Referer: https://evil.com" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "attacker",  
  "email": "evil@test.com",  
  "phone": "999999999",  
  "message": "unauthorized request"  
'
```

```
siva-leadtap@ip-10-0-10-55:~$ curl -X POST https://nilvan.duckdns.org/api/sendgrid \  
-H "Origin: https://evil.com" \  
-H "Referer: https://evil.com" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "attacker",  
  "email": "evil@test.com",  
  "phone": "999999999",  
  "message": "unauthorized request"  
}'  
{ "success": true } siva-leadtap@ip-10-0-10-55:~$ |
```

Microsoft Store

### NILAVAN REALTORS

You have a new contact form submission:

**Name:** attacker

**Email:** [evil@test.com](mailto:evil@test.com)

**Phone:** 999999999

**Message:** unauthorized request

## Result

The request was accepted successfully even though it originated from an untrusted domain.

## Recommended Fix

Implemented trusted origin validation inside:



```
app/api/sendgrid/route.ts
```

Remediation:

```
const origin = req.headers.get("origin");
const referer = req.headers.get("referer");

const allowedOrigin = "https://nilvan.duckdns.org";

if (
  origin !== allowedOrigin ||
  !referer?.startsWith(allowedOrigin)
) {
  return NextResponse.json(
    { error: "Forbidden" },
    { status: 403 }
  );
}
```

## Verification

After remediation:

Requests from untrusted domains returned:

HTTP 403 Forbidden

Only requests originating from the official frontend domain were accepted successfully.

```
siva-leadtap@ip-10-0-10-55: ~$ curl -X POST https://nilvan.duckdns.org/api/sendgrid \
-H "Origin: https://evil.com" \
-H "Referer: https://evil.com" \
-H "Content-Type: application/json" \
-d '{
  "name": "attacker",
  "email": "evil@test.com",
  "phone": "9999999999",
  "message": "unauthorized request"
}'
{"error": "Forbidden Request Origin"}siva-leadtap@ip-10-0-10-55: ~$ |
```

# Vulnerable and Outdated Dependencies

## 5. Vulnerable and Outdated Dependencies

### OWASP Category

A06:2021 – Vulnerable and Outdated Components

### Severity

Critical / High

### Description

The application contains multiple outdated third-party dependencies identified through npm audit. Several installed packages have publicly disclosed security vulnerabilities affecting the application security posture.

### Affected Components

- next
- axios
- lodash
- form-data
- glob
- postcss

### Business Impact

Exploitation of vulnerable dependencies may lead to:

Several attacks

### Proof of Concept

npm audit

```

siva-leadtap@ip-10-0-10-55: ~ - x
+
siva-leadtap@ip-10-0-10-55:~/lt-nilavan$ npm audit
# npm audit report

axios 1.0.0 - 1.15.1
Severity: high
Axios is vulnerable to DoS attack through lack of data size check - https://github.com/advisories/GHSA-4hjh-mcwv-xvwj
Axios has a NO_PROXY Hostname Normalization Bypass that Leads to SSRF - https://github.com/advisories/GHSA-3p68-rcdw-qgx5
Axios has Unrestricted Cloud Metadata Exfiltration via Header Injection Chain - https://github.com/advisories/GHSA-fvcv-3m26-pcqx
Axios: Authentication Bypass via Prototype Pollution Gadget in 'validateStatus' Merge Strategy - https://github.com/advisories/GHSA-w9j2-pvgh-6h63
Axios: Incomplete Fix for CVE-2025-62718 - NO_PROXY Protection Bypassed via RFC 1122 Loopback Subnet (127.0.0.0/8) in Axios 1.15.0 - https://github.com/advisories/GHSA-pmww-cvhr-8vh7
Axios: Invisible JSON Response Tampering via Prototype Pollution Gadget in 'parseReviver' - https://github.com/advisories/GHSA-3w6x-2g7m-8v23
Axios has prototype pollution read-side gadgets in HTTP adapter that allow credential injection and request hijacking - https://github.com/advisories/GHSA-q8qp-cvwc-x6jj
Axios: Null Byte Injection via Reverse-Encoding in AxiosURLSearchParams - https://github.com/advisories/GHSA-xhjh-pmcv-23jw
Axios: CRLF Injection in multipart/form-data body via unsanitized blob type in formDataToStream - https://github.com/advisories/GHSA-445q-vr5w-6q77
Axios: no_proxy bypass via IP alias allows SSRF - https://github.com/advisories/GHSA-m7pr-hjgh-92cm
Axios: unbounded recursion in toFormData causes DoS via deeply nested request data - https://github.com/advisories/GHSA-62hf-57xw-28j9
Axios' HTTP adapter-streamed uploads bypass maxBodyLength when maxRedirects: 0 - https://github.com/advisories/GHSA-5c9x-8gcm-mpgx
Axios: HTTP adapter-streamed responses bypass maxContentLength - https://github.com/advisories/GHSA-vf2m-468p-8v99
Axios: Prototype Pollution Gadgets - Response Tampering, Data Exfiltration, and Request Hijacking - https://github.com/advisories/GHSA-pf86-5x62-jrwf
Axios: Header Injection via Prototype Pollution - https://github.com/advisories/GHSA-6chq-wfr3-2hj9
Axios: XSRF Token Cross-Origin Leakage via Prototype Pollution Gadget in 'withXSRFToken' Boolean Coercion - https://github.com/advisories/GHSA-xx6v-rp6x-q39c
Axios is Vulnerable to Denial of Service via __proto__ Key in mergeConfig - https://github.com/advisories/GHSA-43fc-jf86-j433
fix available via `npm audit fix`
node_modules/axios

brace-expansion 2.0.0 - 2.0.2
Severity: moderate
brace-expansion Regular Expression Denial of Service vulnerability - https://github.com/advisories/GHSA-v6h2-p8h4-qcjq
brace-expansion: Zero-step sequence causes process hang and memory exhaustion - https://github.com/advisories/GHSA-f886-m6hf-6m8v
fix available via `npm audit fix`
node_modules/brace-expansion

follow-redirects <=1.15.11
Severity: moderate
follow-redirects Leaks Custom Authentication Headers to Cross-Domain Redirect Targets - https://github.com/advisories/GHSA-r4q5-vmmm-2653
fix available via `npm audit fix`
node_modules/follow-redirects
```

```

siva-leadtap@ip-10-0-10-55: ~ - x
+
Next.js Improper Middleware Redirect Handling Leads to SSRF - https://github.com/advisories/GHSA-4342-x723-ch2f
Next.js Race Condition to Cache Poisoning - https://github.com/advisories/GHSA-qpvj-v59x-3qc4
Next.js is vulnerable to RCE in React flight protocol - https://github.com/advisories/GHSA-9qr9-h5gf-34mp
Next.js self-hosted applications vulnerable to DoS via Image Optimizer remotePatterns configuration - https://github.com/advisories/GHSA-9g9p-9gw9-jx7f
Authorization Bypass in Next.js Middleware - https://github.com/advisories/GHSA-f82v-jwr5-mffw
Next.js: HTTP request smuggling in rewrites - https://github.com/advisories/GHSA-ggv3-7p47-pfv8
Next.js: Unbounded next/image disk cache growth can exhaust storage - https://github.com/advisories/GHSA-3x4c-7xq6-9pq8
Next.js has a Denial of Service with Server Components - https://github.com/advisories/GHSA-q4gf-8mx6-v5v3
Depends on vulnerable versions of postcss
fix available via `npm audit fix --force`
Will install next@15.5.18, which is outside the stated dependency range
node_modules/next

picomatch <=2.3.1
Severity: high
Picomatch: Method Injection in POSIX Character Classes causes incorrect Glob Matching - https://github.com/advisories/GHSA-3v7f-55p6-f55p
Picomatch has a ReDoS vulnerability via extglob quantifiers - https://github.com/advisories/GHSA-c2c7-rcm5-vvqj
fix available via `npm audit fix`
node_modules/picomatch

postcss <8.5.10
Severity: moderate
PostCSS has XSS via Unescaped </style> in its CSS Stringify Output - https://github.com/advisories/GHSA-qx2v-qp2m-jg93
fix available via `npm audit fix --force`
Will install next@15.5.18, which is outside the stated dependency range
node_modules/next/node_modules/postcss
node_modules/postcss

yaml 2.0.0 - 2.8.2
Severity: moderate
yaml is vulnerable to Stack Overflow via deeply nested YAML collections - https://github.com/advisories/GHSA-48c2-rrv3-qjmp
fix available via `npm audit fix`
node_modules/yaml

11 vulnerabilities (4 moderate, 5 high, 2 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force
```

## Result

11 vulnerabilities detected

2 Critical

5 High

4 Moderate

## Recommended Fix

npm audit fix

npm install next@latest

npm update

## **Verification**

npm audit

Expected Result:

found 0 vulnerabilities